

AccelES: Accelerating Top-K SpMV for Embedding Similarity via Low-bit Pruning

Jiaqi Zhai† Xuanhua Shi*†, Kaiyi Huang†

Chencheng Ye† Weifang Hu† Bingsheng He‡ Hai Jin†

National Engineering Research Center for Big Data Technology and System/Services Computing Technology
and System Lab/Cluster and Grid Computing Lab, School of Computer Science and Technology,

Huazhong University of Science and Technology, Wuhan, 430074, China†

School of Computing, National University of Singapore, 119077, Singapore‡

{jqzhai, xhshi, kyhuang, yecc, huwf, hjin}@hust.edu.cn,

hebs@comp.nus.edu.sg

Abstract—In the realm of recommendation systems, achieving real-time performance in embedding similarity tasks is often hindered by the limitations of traditional *Top-K sparse matrix-vector multiplication* (SpMV) methods, which suffer from high latency due to inefficient memory access patterns. This paper identifies these critical gaps and introduces AccelES, a novel approach that significantly enhances the efficiency of Top-K SpMV. Our method employs a two-stage calculation scheme: the first stage utilizes a compact, low-bit dataset to quickly identify the most relevant entries, while the second stage performs full-precision calculations solely on this pruned subset, thereby minimizing computational overhead. Furthermore, AccelES incorporates innovative matrix representations, Ultra-CSR and Random-CSR, which optimize memory bandwidth utilization. Experimental results demonstrate that AccelES accelerates performance, surpassing state-of-the-art FPGA, GPU, and CPU solutions by factors of $3.4\times$, $2.5\times$, and $153.3\times$, respectively, under controlled conditions. These advancements not only enhance processing speed but also significantly improve real-time performance in recommendation systems, establishing AccelES as a pivotal contribution to the field of Top-K sparse matrix-vector multiplication.

Index Terms—Sparse Matrix-Vector Multiplication, Recommendation Systems, Low-Bit Quantization, Memory Optimization, Accelerator

I. INTRODUCTION

Top-K *sparse matrix-vector multiplication* (SpMV) is a crucial operation that significantly impacts the performance of various applications, including recommendation systems, artificial intelligence, and graph analysis [21], [28], [36], [43]. In practical scenarios, sparse matrices often contain millions of rows, with non-zero matrix elements (NNZ) widely dispersed across memory, as evidenced by previous studies [15], [17], [22], [24], [26], [33]. This distribution leads to substantial memory latency, which can account for 60% to 70% of the overall computation time [39].

Current best practices for Top-K SpMV on CPU and GPU platforms typically involve a two-stage process: performing SpMV followed by sorting operations. However, these traditional methods often face significant limitations. The sorting stage alone can account for over 60% of total

latency, creating a major bottleneck. Consequently, achieving efficient Top-K SpMV computations under the constraints of large-scale streaming and stringent real-time requirements presents a significant challenge for traditional hardware, such as CPUs and GPUs. These inefficiencies have real-world implications—delays in recommendation systems, for example, can reduce user engagement, with studies showing that a one-second delay may lead to a 13% drop in conversions [32]. Similarly, in AI applications, the inability to process large-scale data in real time can increase operational costs and hinder timely insights [46]. Addressing these issues has motivated this research, driving the exploration of innovative methods and architectures.

Field-Programmable Gate Arrays (FPGAs) offer several distinct advantages over CPUs and GPUs for Top-K SpMV. They provide customization and flexibility, enabling efficient resource allocation for the sparse and irregular patterns of SpMV tasks. FPGAs also exploit fine-grained parallelism more effectively than traditional hardware, handling irregular memory access patterns with greater efficiency [18]. The fine-grained memory access control and high transmission bandwidth of multi-channel *High Bandwidth Memory* (HBM) improve memory throughput and reduce latency [37]. Moreover, FPGAs can be more energy-efficient, consuming less power than CPUs and GPUs for the same tasks, which is beneficial for power-constrained applications. The inherent parallel architecture of FPGAs allows for scalable performance, accommodating increases in dataset size or computational complexity without requiring new hardware [9]. These advantages enable FPGAs to deliver superior computational performance and power efficiency, making them well-suited for the specialized workloads of Top-K SpMV.

Current FPGA-based Top-K SpMV accelerator [25] continues to use optimization methods for the SpMV operator, typically using floating-point (FP32) data to perform SpMV calculations on the entire sparse matrix. Although some studies have introduced quantization techniques to improve bandwidth utilization, they maintain relatively high bit-width, such as 20 bits. Such a strategy using high bit-width data brings about a significant bottleneck: within a single clock cycle,

*: Xuanhua Shi is the corresponding author.

the 512-bit HBM bus accommodates at most 15 non-zero elements [25]. Many research works use traditional sparse matrix representations such as Coordinate (COO) or *Compressed Sparse Row* (CSR) [19], [30], [31]. The COO format does not compress the coordinates of non-zeros, requiring a substantial amount of memory space and resulting in extremely inefficient bandwidth utilization. Although the CSR format compresses the row coordinates of non-zero elements, the original row pointers format is only suitable for row-level parallelism, missing the opportunity for speedup at the non-zero element level. While adding a row pointer for each non-zero in the CSR format allows for parallel processing at the granularity of non-zeros, like BS-CSR [25], it results in wasted transmission bandwidth, exacerbating the issue of low bandwidth utilization. In summary, inefficient memory transfer remains a critical bottleneck limiting the performance of Top-K SpMV acceleration systems. Optimizing memory transfer mechanisms and data storage formats is crucial for further enhancing acceleration performance.

While prior SpMV acceleration approaches gained substantially on traditional hardware, we find that they can be further optimized as their computation with the full-precision data is unnecessary in the scenario of Top-K SpMV applications, as there is no need for performing full-precision calculation for the rows does not produce Top-K results nor the elements less significantly affect the Top-k results. Therefore, the approaches introduce significant redundancy. We classify the redundancy into two categories. **Rows Redundancy:** The number of rows in a sparse matrix typically far exceeds K. If we can pre-identify the rows that will yield the Top-K results, we can transmit only the non-zeros of these Top-K rows to the accelerator, reducing unnecessary data transfer. **Non-zeros Redundancy:** The absolute values of non-zeros in a sparse matrix vary greatly. Elements with minimal absolute values contribute little to the final sorting results. Thus, transmitting these low-impact elements to the accelerator is unnecessary.

To reduce the redundancy, we propose a novel two-stage Top-K SpMV algorithm that minimizes the memory accesses and excessive computation on less significant data. Initially, we perform full-scale Top-K calculations on a smaller dataset quantized to 6-bit integers. Based on the Top-K results from this dataset, we prune the full-precision dataset. Subsequently, we conduct full-precision Top-K calculations only on the pruned dataset. This algorithm is implemented in a two-stage Top-K SpMV accelerator with a heterogeneous core architecture named AccelES. AccelES leverages the high bandwidth of HBM, utilizing multiple Q-cores in the first stage and a single F-core in the second stage. By reducing the bit-width of non-zeros, we enhance the parallelism of calculations and improve data transmission. We also introduce two sparse matrix formats, Ultra-CSR and Random-CSR, designed to maximize bit-width utilization and alleviate memory burden during streaming and random processing stages. Ultra-CSR optimizes HBM bit-width usage by representing row indices of non-zeros in the sparse matrix with a single bit. Random-CSR employs a metadata approach to store row indices

alongside their base addresses, facilitating efficient random access to row elements. To address redundancy, we propose a generalized ReSparse algorithm that zeroes out non-zeros in sparse matrices contributing minimally to the ranking, thereby reducing memory accesses.

Experiments on five real-world and six synthetic workloads demonstrate that AccelES achieves speeds that are $3.3\times$, $2.4\times$, and $153.3\times$ faster than state-of-the-art FPGA, GPU, and CPU solutions, respectively. Additionally, AccelES reduces row redundancy by nearly 100% and non-zero redundancy by 31.1%, while cutting memory accesses by 73.5%. Despite these substantial gains, AccelES maintains a minimal accuracy loss of just 0.95%.

Overall, our contributions are as follows:

- We propose an innovative Top-K SpMV algorithm utilizing low-bit pruning with 6-bit quantization, effectively addressing memory access bottlenecks and real-time computation challenges.
- We design two sparse matrix formats, Ultra-CSR and Random-CSR, to maximize HBM bit-width utilization during streaming and random access phases while minimizing memory overhead.
- We introduce a ReSparse algorithm to eliminate non-zeros in sparse matrices that contribute minimally to the sorting process, thereby reducing the number of non-zeros that need to be transmitted.
- We develop a heterogeneous architecture, AccelES, based on FPGA with HBM, for the Top-K SpMV algorithm, ensuring efficient memory utilization and high throughput.

II. RELATED WORK

Multi-threaded CPU designs, such as *sparse_dot_topn* [1], [34], [38], enable parallel Top-K SpMV computation. However, these implementations face significant challenges due to limited memory bandwidth and insufficient cache sizes, which hinder their performance in handling extensive random access patterns typical in SpMV operations. While optimizations for general SpMV have been explored, they often overlook the specific requirements of Top-K SpMV, leading to substantial performance bottlenecks. Significant efforts have been made to optimize SpMV and SpMM on GPUs [5], [12], [14], [41], particularly with libraries like *cuSPARSE* [23]. However, the inherent discrepancy between the GPU memory hierarchy and the irregular access patterns of sparse matrices hampers their efficiency for Top-K operations. The thread synchronization overhead and inefficient memory bandwidth utilization further exacerbate these issues.

FPGA-based accelerators [11], [13] offer a promising alternative due to their configurable computing modules and optimized data paths. The Vitis_Library [2] provides a benchmark for the SpMV operator of the Vitis SPARSE library, which can run on the Alveo U280. FSPA [44] combines CAM arrays with MAC arrays to enhance SpMV performance. However, it utilizes the COO format, which has low transmission efficiency and fails to optimize for Top-K operations. Serpens [30] and Sextans [31] adopt a memory-centric design utilizing HBM

and on-chip URAM to accelerate SpMV and SpMM operations, respectively. While they leverage the high bandwidth of HBM, the use of FP32 data in COO format creates a significant transmission bottleneck. Moreover, directly applying the optimization methods for SpMV and SpMM to the Top-K problem leads to wasted on-chip resources and introduces additional sorting overhead. Parravicini et al. [25] design a multi-core parallel approximate computing architecture and a streaming sparse matrix format, BS-CSR, for the Top-K SpMV operator on FPGAs equipped with HBM. However, the limited bit-width of HBM restricts the amount of FP32 data that can be transmitted per clock cycle, thus becoming a performance bottleneck.

The limitations identified in the existing literature directly motivate our research objectives. Our proposed heterogeneous architecture, AcceLES, introduces innovative low-bit pruning algorithms and novel matrix representations, namely Ultra-CSR and Random-CSR, which specifically target the unique challenges of Top-K SpMV operations. These innovations not only enhance memory efficiency but also significantly reduce computational complexity compared to existing methods.

III. TOP-K SPMV VIA LOW-BIT PRUNING

A. Top-K SpMV Workflow

Given a sparse matrix $\mathbf{A}$ of size $M \times N$ and a dense vector $\mathbf{x}$ of size $N \times 1$, the objective of Top-K SpMV is to compute the dot product $\mathbf{y} = \mathbf{A} \cdot \mathbf{x}$ and return the top K highest values in the $M \times 1$ vector $\mathbf{y}$, as described in algorithm 1. In other words, Top-K SpMV identifies the top K rows in $\mathbf{A}$ that have the highest inner product similarity with $\mathbf{x}$. Each row in the sparse matrix composed of embeddings contains non-zero elements. In the context of Sparse Embedding, M typically ranges in the millions, N is usually in the hundreds [16], [20], and K is generally within 100 [40].

Algorithm 1: Top-K SpMV

Input : Sparse matrix $\mathbf{A}$, dense vector $\mathbf{x}$, number of top elements k

Output: Top-K values **topK_values**, indices of Top-K values **topK_indices**

```

1 Function TOP-K-SpMV ( $\mathbf{A}, \mathbf{x}, k$ ) :
2    $\mathbf{y} \leftarrow \text{zeros}(\text{length of } \mathbf{A}.\text{rows});$ 
3   for each non-zero element  $(i, j, A_{ij}) \in \mathbf{A}$  do
4      $\mathbf{y}[i] \leftarrow \mathbf{y}[i] + A_{ij} \times \mathbf{x}[j];$ 
5    $[\text{topK\_values}, \text{topK\_indices}] \leftarrow \text{TopK}(\mathbf{y}, k);$ 
6   return topK_values, topK_indices;

```

B. Dataflow with Sparse Matrix Low-bit Quantization

To address the **Rows Redundancy** of communication and computation, we design a Top-K SpMV via a low-bit pruning algorithm, as illustrated in Fig.1. Initially, $\mathbf{A}$ is quantized and converted into 6-bit integers, referred to as $\mathbf{Q}_A$, and subsequently stored in memory. This quantization step is performed only once. Subsequently, the following established

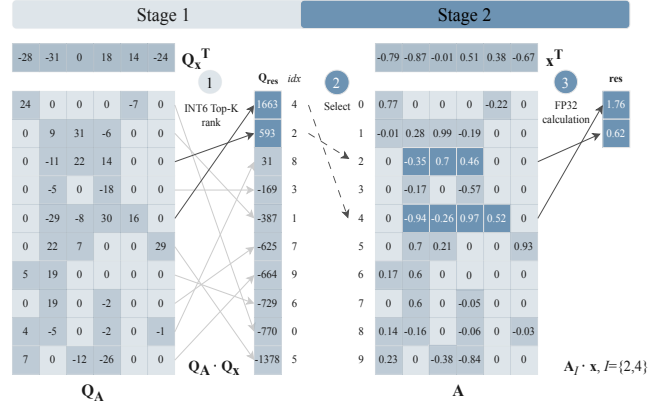


Fig. 1. Top-K SpMV with low-bit pruning

calculation process will be followed for processing continuous query vector inputs. In stage 1, the algorithm employs the quantized $\mathbf{Q}_A$ and $\mathbf{Q}_x$ to compute the SpMV results for all rows. Simultaneously, the algorithm uses an in-place comparison strategy to count the Top-K results. In stage 2, the FP32 data of $\mathbf{A}_I$ corresponding to the Top-K result row indices are retrieved from memory to compute the precise Top-K results. In this example, we only require the Top-2 FP32 results. Using quantized data for computation provides three distinct advantages: 1) Quantized data occupies less bit-width, alleviating the pressure on memory transmission; 2) Some non-zeros are quantized to zero (e.g. the element at position (1,0) in $\mathbf{A}$ is quantized from -0.01 to 0), further alleviating the pressure on memory transmission; 3) Since we use integer quantization, integer computation consumes fewer DSPs to floating-point computation.

We employ a linear quantization method to convert the FP32 data into B-bit integer values, which involves scaling all elements in $\mathbf{A}$ and $\mathbf{x}$ by a constant factor and then rounding and clipping them to fit within the range of $[-2^{B-1}, 2^{B-1} - 1]$. The quantization formulas are as follows:

$$S = \frac{2^{B-1}}{\max(|\mathbf{A}_{\max}|, |\mathbf{A}_{\min}|)} \quad (1)$$

$$\mathbf{Q}_A = \text{clip}(\text{round}(S \times \mathbf{A}), -2^{B-1}, 2^{B-1} - 1) \quad (2)$$

The dense vector $\mathbf{Q}_x$ undergoes quantization using the same method. Linear quantization offers high precision and low computational complexity but introduces noise. It scales each vector equally, ideally preserving vector directions and inner product similarity rankings. However, $\text{round}()$ converts scaled floats to the nearest integers, introducing noise, while $\text{clip}()$ confines values within a range, causing further non-uniform scaling. Consequently, vector directions deviate slightly, resulting in a loss of precision.

Fig.1 illustrates that the results after low-bit quantization maintain the same ranking order as the FP32 results, which is an ideal scenario. Unfortunately, perfect accuracy is achievable

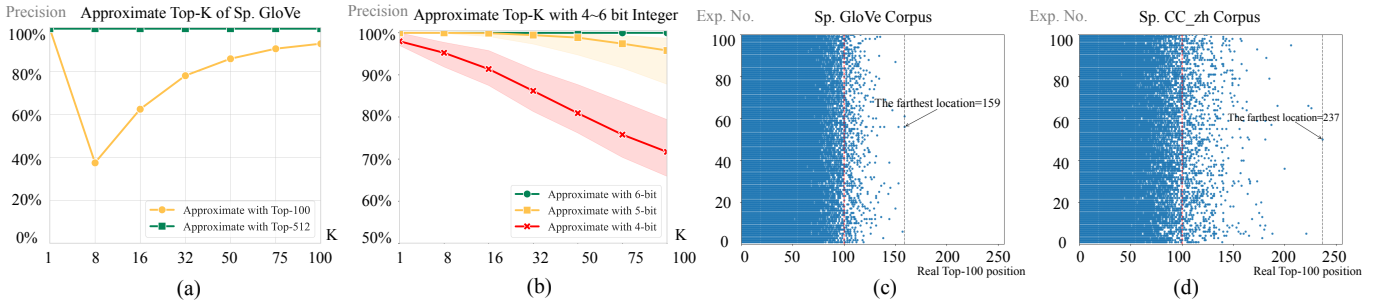


Fig. 2. Approximate range guidance for Top-K SpMV using a low-bit dataset. (a) The approximation accuracy for the GloVe corpus with Top-100 and Top-512 low-bit dataset results. (b) The approximate accuracy confidence intervals in various datasets using 4- to 6-bit integer quantization. (c) Deviation cases in the GloVe dataset. (d) Deviation cases in the CC_zh corpus.

only when dealing with sparse matrices with few rows. GloVe (*Global Vectors of Word Representations*) [26] and CC_zh [15] are widely used word vector representations, containing 0.4 and 2 million rows of embeddings respectively. We employ the sparse embedding techniques proposed by [10] to transform them into sparse matrices. The yellow line in Fig.2 (a) illustrates that directly approximating the Top-100 statistics using quantized GloVe data results in a significant error. However, when approximating the Top-100 statistics using a larger range of 512, there is no loss in accuracy. Fig.2 (b) illustrates the impact of 4 to 6-bit quantization on accuracy. We observe that 6-bit quantization has no impact on ranking accuracy, 5-bit quantization results in up to a 10% accuracy loss, and 4-bit quantization causes a maximum accuracy loss of 32.5%. Fig.2 (c) focuses on identifying and analyzing the most deviated cases in the Sparse GloVe dataset. This analysis is pivotal for validating the reliability of our approach in practical scenarios. Fig.2 (d) extends the analysis to a large-scale sparse matrix with 2 million rows. It showcases that despite the increased complexity and data volume, our low-bit pruning method consistently captures the true Top-100 results within the approximation range. This scalability is essential for handling real-world data sizes in high-performance computing applications. Overall, Fig.2 underscores the balance between computational efficiency and ranking precision achieved through our quantization method, thereby justifying its integration into the Top-K SpMV algorithm for large-scale and precise computations.

C. ReSparse Algorithm

To address **Non-zeros Redundancy** in communication and computation, we propose the ReSparse algorithm. ReSparse optimizes Top-K SpMV operations by selectively zeroing out non-zeros within the sparse matrix, particularly those with values near zero, which have minimal impact on ranking results, as depicted in Fig.3. This reduction minimizes redundancy in both communication and computation, enhancing efficiency during sparse matrix operations. Specifically, for each row vector in A , the algorithm ranks the absolute values of its non-zeros. It retains elements within the top keep percent (kp) and

sets the others to zero. The top keep percent (kp) refers to the proportion of the largest absolute value elements in each row that are preserved, ensuring that only the most significant values are kept. Algorithm 2 details the ReSparse algorithm.

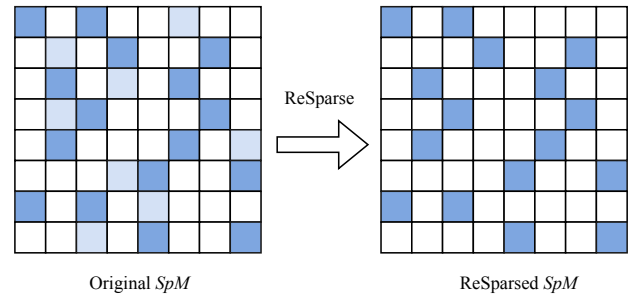


Fig. 3. ReSparse on sparse matrix

In Algorithm 2, kp serves as a pivotal hyperparameter controlling the effectiveness of the ReSparse algorithm. As kp increases, fewer elements are zeroed out, thereby enhancing computational precision by preserving more information from the original A . To balance precision loss and leverage the benefits of ReSparse, Algorithm 3 employs a binary search method to determine the optimal value of kp for a specified sparse matrix A . We search kp on the sparse matrix that we pre-sorted by absolute value, eliminating the need for repeated sorting operations during subsequent searches. To comprehensively evaluate precision across varying K values in Top-K SpMV computations, we introduce *keyMetric* as a unified assessment metric. It consolidates precision scores across varying K values through weighted adjustments, ensuring fair evaluation that accommodates diverse application scenarios.

Algorithm 3 utilizes a binary search strategy to iteratively adjust the current ReSparse ratio *curPct*, applying ReSparse and quantization operations to the original sparse matrix. The ReSparse ratio starts at 1.0 in the first iteration, and *accThresh* is established based on the user-defined tolerance for precision loss. In each subsequent iteration, we calculate the proportion of true Top-K indices that are encompassed by

Algorithm 2: Resparse Sparse Matrix

Input : Sparse matrix $\mathbf{A}$, keep percentage kp
Output: Resparsed sparse matrix $\mathbf{A}_{\text{rsp}}$

```

1 Function RESPARSE-SPM( $\mathbf{A}, kp$ ):
2    $rows, cols \leftarrow \mathbf{A}.shape$ ;
3    $avgAbsVal \leftarrow \text{mean}(\text{abs}(\mathbf{A}))$ ;
4    $\mathbf{A}_{\text{rsp}} \leftarrow \text{zeros\_like}(\mathbf{A})$ ;
5   for  $row \leftarrow 0$  to  $rows - 1$  do
6      $rowData \leftarrow \mathbf{A}[row, :]$ ;
7      $nnzIdx \leftarrow \text{nonzero}(rowData)[0]$ ;
8      $nnzVal \leftarrow rowData[nnzIdx]$ ;
9     if  $\min(\text{abs}(nnzVal)) > avgAbsVal$  then
10       $\mathbf{A}_{\text{rsp}}[row] \leftarrow rowData$ ;
11   else
12      $sortedIdx \leftarrow \text{argsort}(\text{abs}(nnzVal))$ ;
13      $keepNum \leftarrow \text{len}(sortedIdx) \times kp$ ;
14     for  $idx \in sortedIdx[:keepNum]$  do
15        $\mathbf{A}_{\text{rsp}}[row, nnzIdx[idx]] \leftarrow nnzVal[idx]$ ;
16 return  $\mathbf{A}_{\text{rsp}}$ ;
```

the approximate Top-512 set, denoted as $prec$. Subsequently, we assess the convergence of the process by employing the weighted precision metric $keyMetric$. If the $curPct$ matches a previously searched value, the algorithm terminates and returns the most recent kp from the previous iteration that satisfies the condition $keyMetric > keyThresh$. This iterative approach ensures that the algorithm maximizes the ReSparse ratio while maintaining accuracy, thereby reducing the number of non-zeros that need to be passed to the accelerator.

Assume the true computational result for the i -th row is $y_i = \mathbf{A}_i \cdot \mathbf{x}$. After applying the ReSparse method, the computed result becomes $\hat{y}_i = \hat{\mathbf{A}}_i \cdot \mathbf{x}$, where $\hat{\mathbf{A}}_i$ is the sparsified row vector retaining $kp \times nnz(\mathbf{A}_i)$ non-zeros. If the largest absolute value among the zeroed-out elements is M and the largest absolute value in $\mathbf{x}$ is $\|\mathbf{x}\|_\infty$, the worst-case computational error for the i -th row is:

$$\begin{aligned} \delta_i &= y_i - \hat{y}_i = \sum_{j \notin S'} \mathbf{A}_{ij} \mathbf{x}_j \\ &\leq (1 - kp) \cdot nnz(\mathbf{A}_i) \cdot M \cdot \|\mathbf{x}\|_\infty \end{aligned} \quad (3)$$

Where S' represents the set of indices of the retained non-zeros in $\hat{\mathbf{A}}_i$. To guarantee the accuracy of the sorting results, the following condition must be satisfied:

$$y_K - \delta_i \geq y_{K+1} \quad (4)$$

In this context, y_K signifies the minimum value within the Top-K ranked results, while y_{K+1} denotes the maximum value among the results not included in the Top-K. Fine-tuning the retention ratio kp allows for managing sorting inaccuracies introduced by the ReSparse algorithm, ensuring

Algorithm 3: Search for Optimal ReSparse Ratio

Input : Sparse matrix $\mathbf{A}$, vector $\mathbf{x}$, accuracy threshold $accThresh = 0.9999$

Output: Optimal ReSparse ratio

```

1 Function SEARCH-RESPARSE-RATIO( $\mathbf{A}, \mathbf{x}, accThresh$ ):
2    $high \leftarrow 1.0$ ;
3    $low \leftarrow 0.0$ ;
4    $W \leftarrow [1, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{50}, \frac{1}{75}, \frac{1}{100}]$ ;
5    $epoch \leftarrow 0$ ;
6    $oldPercents \leftarrow ()$ ;
7   while true do
8     if  $epoch = 0$  then
9        $curPct \leftarrow 1.0$ ;
10    else
11       $curPct \leftarrow \text{round}((low + high)/2, 3)$ ;
12       $\mathbf{A}_{\text{rsp}} \leftarrow \text{RESPARSE-SPM}(\mathbf{A}, curPct)$ ;
13       $\mathbf{Q}_A, \mathbf{Q}_x \leftarrow \text{quantify}(\mathbf{A}_{\text{rsp}}, \mathbf{x})$ ;
14       $oldPercents.add(curPct)$ ;
15       $prec \leftarrow \text{cpt\_precision}(\mathbf{A}, \mathbf{x}, \mathbf{Q}_A, \mathbf{Q}_x)$ ;
16       $keyMetric \leftarrow 0$ ;
17      for  $i \leftarrow 0$  to  $\text{len}(W)$  do
18         $keyMetric \leftarrow keyMetric + prec[i] * W[i]$ ;
19      if  $epoch = 0$  then
20         $keyThresh \leftarrow keyMetric * accThresh$ ;
21      else
22        if  $keyMetric > keyThresh$  then
23           $high \leftarrow curPct$ ;
24        else
25           $low \leftarrow curPct$ ;
26        if  $curPct \in oldPercents$  then
27          return  $high$ ;
28       $epoch \leftarrow epoch + 1$ ;
```

they meet specific application demands. The ablation study results, detailed in Section VII-B, show that ReSparse achieves an average rate of setting non-zero elements to zero of 35.7% across five publicly sparse corpora and six large-scale synthetic sparse matrices, while keeping accuracy loss within 0.87%.

IV. ARCHITECTURE OVERVIEW

We introduce a heterogeneous architecture named AccelES. The primary objective of AccelES is to enhance computational efficiency while maintaining high accuracy in retrieving top results from large-scale sparse matrices. AccelES comprises 32 Q-core units for processing low-bit quantized matrices to guide the pruning of full-precision matrices, and 1 F-core unit for calculating Top-K results on the pruned full-precision matrices, as illustrated in Figure 4. We propose two sparse matrix layouts, Ultra-CSR and Random-CSR, to optimize data storage and access efficiency. This Section provides an overview of AccelES. In Sec.V and Sec.VI, we

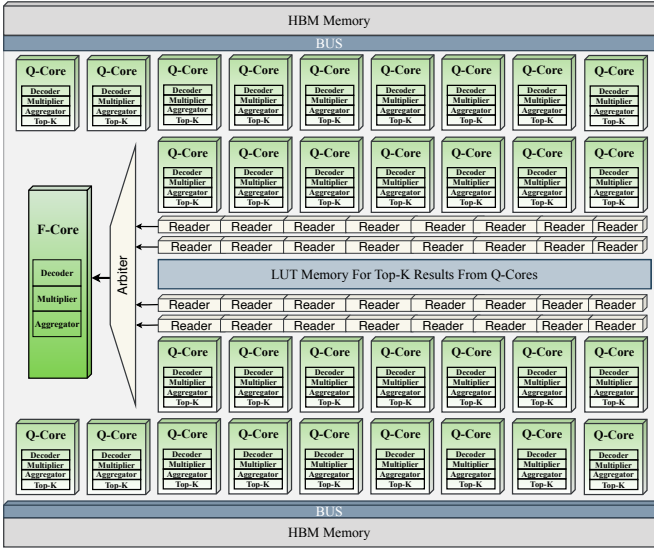


Fig. 4. The overall architecture of AccelES

detail the proposed sparse matrix layouts and the design of the heterogeneous computation cores, respectively.

AccelES performs parallel computation at the non-zero element level. The number of non-zeros transmitted through the HBM bus per cycle determines the system's maximum computational throughput. To improve computing performance, we design Q-core by referencing the multi-core parallel computing architecture from [25], interconnecting 32 Q-cores with 32 HBM channels. We partition the sparse matrix $\mathbf{Q}_A$ into 32 segments. Each Q-core then performs Top-16 SpMV operations on the vector $\mathbf{Q}_x$ using one segment of $\mathbf{Q}_A$ and identifies the Top-16 row indices. Based on these row indices, the algorithm extracts FP32 data from the corresponding HBM channels. The Arbiter integrates the 32 streams of FP32 non-zeros data, which are then sequentially input to the F-core for FP32 computation, resulting in the final Top-512 FP32 results. To prevent Top-K row clustering and maintain approximation accuracy after partitioning, we randomly shuffle the embedding matrix by row during preprocessing. Shuffling is widely regarded as an effective method for avoiding uneven load distribution and extreme data imbalances in parallel processing scenarios [7]. The accuracy loss caused by multi-core approximation is fully discussed in [25], and we also analyze it through ablation experiments in Sec.VII.

V. SPARSE MATRIX LAYOUT

A. Drawbacks of COO and CSR

When processing large-scale sparse matrices, the memory access bottleneck becomes particularly pronounced due to the vast data accesses to non-zero elements and their random distribution in memory [39]. Although well-designed sparse matrix layouts such as BEICSR [42] and TileCSR [45] can optimize data layout and significantly improve sparse matrix access efficiency in specific workflows, they are not suitable for our scenario. The COO and CSR formats are currently the two most common layouts for sparse matrices. The COO

format utilizes 32 bits each to represent the x and y coordinates, as well as the non-zero value val . It is encoded at the granularity of non-zeros and exhibits no data dependency, making it highly suitable for accelerating Top-K SpMV operations through parallelism and pipelining techniques at the non-zero level. However, each non-zero necessitates 96 bits, which means the Alveo U280's HBM bus can only simultaneously transfer five non-zeros stored in COO format at its 512-bit bit-width, as illustrated in Fig.5.

Although the CSR format improves storage efficiency by compressing non-zero row pointers, it is based on row compression rather than element position compression. This means elements can only be decompressed into the (x, y, val) format at row granularity, missing the opportunity for parallel acceleration at the non-zero element granularity. The introduction of BS-CSR [25] effectively addresses this issue. However, this method leads to redundancy in the row pointers array.

B. Ultra-CSR

The density of sparse embeddings generally exceeds 1%, typically ranging from 2% to 30%. For example, consider a sparse matrix with 500 columns and a sparsity of 2%, as shown in Fig.5. In this case, each row has approximately 10 non-zeros. Using the CSR format, 10 bits represent the y coordinate, and 32 bits represent the non-zero value val . The parameter P , which represents the maximum number of non-zeros that a packet, defined as the 512-bit bit-width of the HBM bus, can accommodate, must satisfy the condition $(10 + 32 + \lceil \log_2 P \rceil) \times P \leq 512$, resulting in $P = 11$ with each ptr occupying 4 bits. Compared to the COO format, the CSR format enhances the throughput capacity of a packet to transmit 11 non-zero elements concurrently in a single cycle.

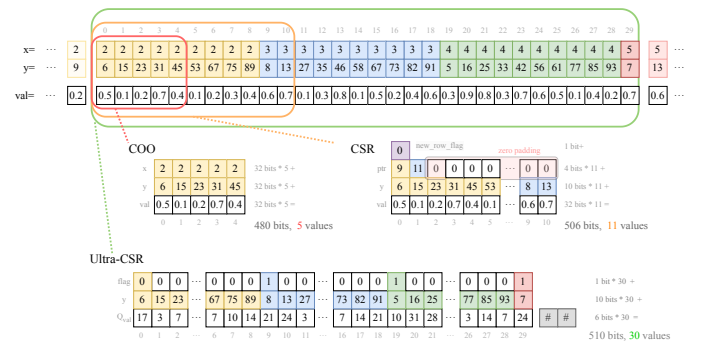


Fig. 5. The Ultra-CSR format enables the HBM 512-bit width to carry 30 elements, 2.7 $\times$ that of the CSR format and 6 $\times$ that of the COO format.

However, we observe that among the 11 non-zero values in a packet, there are at most two rows of elements, resulting in the ptr array having only two valid elements. The remaining 9 elements are filled with zeros, resulting in a waste of 36 bits of width. When directly applying the quantized design to the CSR format, the bit-width waste due to zero-padding becomes more pronounced: Using 6 bits to represent the quantized non-zero value Q_{val} , P must satisfy the condition $(10 + 6 + \lceil \log_2 P \rceil) \times P \leq 512$, resulting in $P = 24$, with each ptr occupying 5 bits.

Row wrapping typically occurs at most three times among the 24 non-zeros in a single packet, indicating that the ptr array contains only three valid entries. Consequently, zero-padding wastes 105 bits, and an additional 8 bits remain idle, resulting in a total bit-width waste of 113 bits. This waste accounts for 22.1% of the total HBM bus bit-width. Effectively utilizing this portion of the bit-width is crucial for mitigating memory access bottlenecks.

We propose the Ultra-CSR sparse matrix format to address this issue, effectively reducing the bit-width waste problem inherent in the CSR format. The Ultra-CSR format uses 10 bits to represent the y coordinate and 6 bits to represent the quantized non-zero value Q_{val} . Each non-zero utilizes 1 bit to indicate whether it transitions to a new row compared to the previous non-zero entry. Since each row of the sparse matrix composed of sparse embedding contains non-zeros, this row compression expression can be correctly parsed to (x, y, Q_{val}) . This design makes the CSR format regular in non-zero granularity, which is conducive to parallel processing and enables a single packet to transmit 30 non-zero values simultaneously. It enhances transmission capacity by $2.73\times$ relative to the design outlined in [25] and by $6\times$ compared to the COO format.

C. Random-CSR

Unlike the streaming access pattern of the quantized matrix $\mathbf{Q}_A$ in the first stage, the pruned FP32 sparse matrix in the second stage is accessed randomly by row. However, COO, CSR, and Ultra-CSR formats all require sequential row access and do not support random access, making full-precision matrix access highly inefficient. Moreover, the bit-width of the FP32 sparse matrix elements is higher. While formats like CSR can compress row information, the limited capacity of HBM memory necessitates a more highly compressed sparse matrix format to accommodate billions of non-zero elements.

Random-CSR

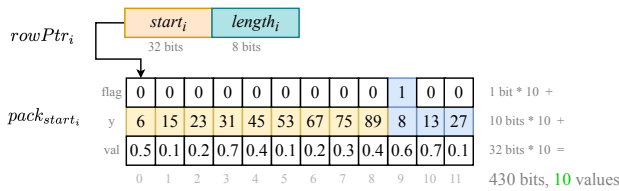


Fig. 6. The Random-CSR format, similar to the meta-data format, allows efficient row-wise random access to matrices.

To overcome these challenges, we design a format for random access of the FP32 sparse matrix, named Random-CSR, as illustrated in Fig.6. It uses a metadata storage format comprising two arrays of structs: $rowPtr$ and $pack$. When accessing data randomly, the $rowPtr$ structure functions like a page table. In $rowPtr_i$, the fields $start_i$ and $length_i$ represent the packet number of the first non-zero in the i -th row and the number of packets occupied by the non-zeros in that row, respectively. The design of the $pack$ structure is similar to

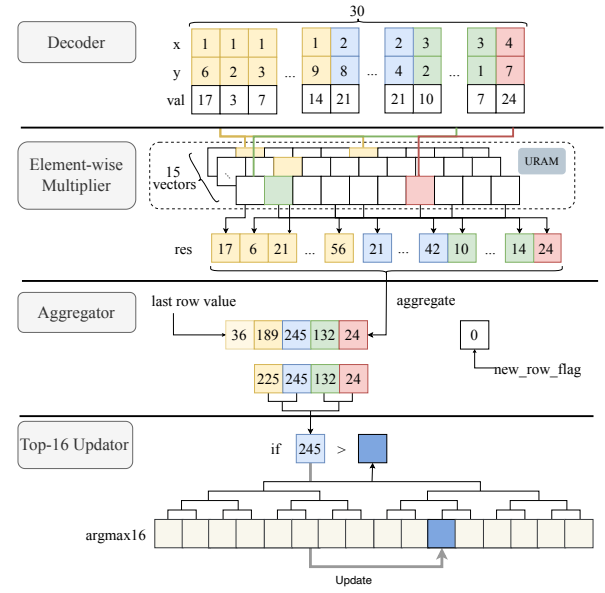


Fig. 7. The Q-core architecture performs fully pipelined parallel processing at the granularity of INT-6 non-zeros.

Ultra-CSR. Each packet contains 10 non-zeros. Each non-zero utilizes 1 bit to denote whether the element has transitioned to a new row compared to the previous non-zero entry. It uses 10 bits to represent the y coordinate and 32 bits to represent the non-zero val . $pack_{start_i}$ contains the first non-zero of the i -th row.

When accessing the FP32 sparse matrix randomly, we first read the i -th element from the $rowPtr$ array. Using the values $start_i$ and $length_i$, we then read the corresponding packets from $pack_{start_i}$ to $pack_{start_i+length_i}$. Once these packets are retrieved, the Reader and Decoder modules, as detailed in Section VI, handle their parsing and processing.

VI. HARDWARE ARCHITECTURE DESIGN

AccelES comprises heterogeneous units Q-core and F-core, which are designed to be fully optimized for non-zero-level parallelism. For each computation core, after reading a data packet containing n non-zeros from HBM, it processes these n elements in parallel during any of the subsequent stages of decoding, computation, and output. This design enhances computational parallelism and addresses the load imbalance caused by the uneven distribution of non-zeros when parallel at the row level [31].

A. Q-core

Q-core is the quantized computation core in the heterogeneous architecture of AccelES, as illustrated in Fig.7. Each Q-core identifies the Top-16 row indices from the SpMV results of $\mathbf{Q}_{A_i}$ and $\mathbf{Q}_x$ to guide the pruning of the full-precision matrix $\mathbf{A}$, where $\mathbf{Q}_{A_i}$ is part of $\mathbf{Q}_A$. The architecture of Q-core encompasses essential components such as the Decoder, the Element-wise Multiplier, the Aggregator, and the Top-16 Updater.

Decoder performs essential preprocessing by decomposing (x, y, val) from Ultra-CSR-encoded data packets. The Ultra-CSR format introduces data dependencies during decoding, where the x coordinate of a non-zero depends on the previous one, complicating parallel decoding of 30 elements. To overcome this challenge, we utilize a tree-structured adder to optimize long dependencies and reduce the addition stage to $\log_2 n$, thereby achieving the ability to parse the x -coordinates of 30 elements simultaneously in a single clock cycle.

Element-wise Multiplier conducts element-wise multiplication operations between Q_x and the non-zeros extracted from data packets. To perform random accesses of high-reuse Q_x in a single clock cycle, we preload it into UltraRAMs (URAMs). Each data packet contains 30 non-zeros. However, each URAM has only two read ports, limiting parallel access capability. To address this limitation, we replicate Q_x 15 times in URAM, enabling simultaneous parallel access. The Alveo U280 has 960 URAMs, each 4096×72 bits, totaling 270MB. In our case, Q_x can be up to 1024×6 bits, fitting within a single URAM. Replicating it 15 times across 32 cores without merging would require 50% of the total URAMs in the Alveo U280.

Aggregator consolidates the results of element-wise multiplications for elements within the same row. During processing, the Aggregator retains the aggregated result of the last row from the previous data packet. It examines the *new_row_flag* indicator of the first element in the new packet to ascertain whether to aggregate across both packets.

Top-16 Updater compares the current Top-16 results of Q-core with the aggregated results from the new data packet to update the Top-16 optimal row indices. Each core on our FPGA maintains a small maximum heap implemented with LUTs. We have designed a series of merge comparison circuits to identify the minimum value in each clock cycle and compare it with incoming values. If a new value exceeds the current minimum in the heap, we update the value in place, which effectively mitigates the intensive data movement typically associated with sorting algorithms. Once the calculation is complete, the Q-core counts the Top-16 results for the sub-matrix it is responsible for. When K exceeds 100 and the accuracy of the Top-512 approximation drops, the Top-16 Updater can quickly adjust to the Top-32 Updater or higher by modifying parameters.

B. F-core

As the FP32 computational core within AccelES, F-core extracts FP32 data of the pruned full-precision sparse matrix from the HBM based on the Top-512 row indices output by Q-cores and performs FP32 SpMV calculations. The architecture of F-core encompasses essential components such as the Decoder, the Element-wise Multiplier, and the Aggregator, as illustrated in Fig.8. In addition, F-core requires 32 independent Readers to parse packets and an Arbiter to arbitrate the data stream.

Reader units are configured in 32 instances, each utilizing the Top-16 row index output by a Q-core to read the FP32

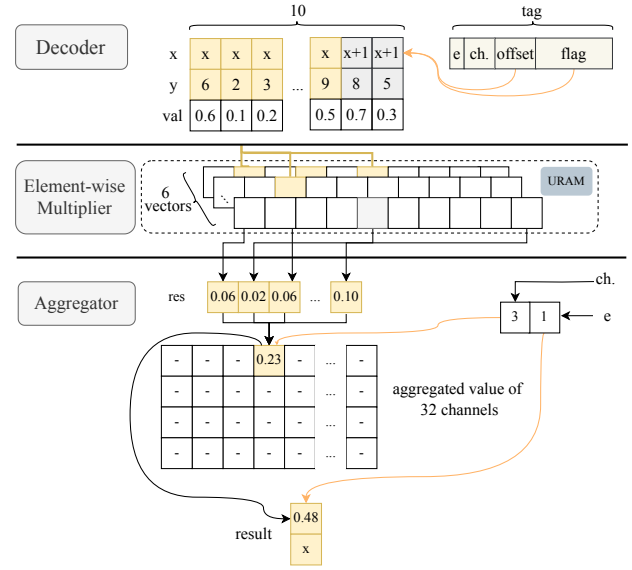


Fig. 8. The F-core architecture performs fully pipelined parallel processing at the granularity of FP32 non-zeros.

sparse matrix stored in Random-CSR format from a specific channel. Once the Reader retrieves the required packets, it repackages them as shown in Fig.9, incorporating the necessary *tag* information for subsequent decoding. The *tag* comprises three fields: *channel*, *offset*, and *end*. The *channel* field uses 5 bits to identify the HBM channel from which the data packet originates, the *offset* field uses 4 bits to indicate the relative position of the first element of the i -th row in the data packet, and the *end* field uses 1 bit to signify whether the i -th row has been fully read.

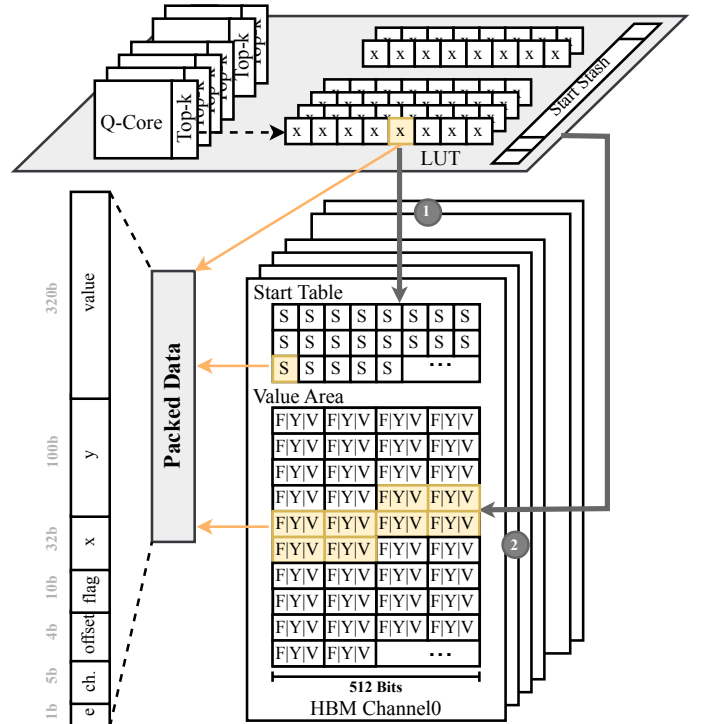


Fig. 9. An example diagram of the Reader workflow

Arbiter is responsible for the arbitration and scheduling of the FP32 data streams. Employing an efficient round-robin scheduling algorithm, the Arbiter seamlessly integrates FP32 data packets from 32 HBM channels, transforming them into a coherent and continuous data stream. This consolidated stream is subsequently directed to the F-core for precise FP32 SpMV computation processing.

Decoder parses the data packets read and compressed by the Reader, extracting the fields *tag*, *x*, *y*, and *value*. Since the position of the first floating-point element of the *i*-th row in the initial data packet is uncertain, we use *new_row_flag* and *offset* to determine from which element in the data packet to start parsing. The Aggregator will use *channel* and *end* fields later.

Element-wise Multiplier shares the same design philosophy as Q-core, with the distinction that the multiplication units in F-core optimize SpMV performance by copying the FP32 $\times$ 5 times in URAM, ensuring parallel random access for 10 non-zeros.

Aggregator aggregates the inner product results of elements in the same row. Using the *channel* field parsed by the Decoder, it maps the multiplication results to corresponding variables for accumulation. Since a single channel processes multiple rows of data, the *end* field helps identify line breaks within each channel.

VII. EVALUATION

A. Experimental Setup

We develop AccelES using *High-Level Synthesis* (HLS) with the Vitis 2023.2 toolchain on the Alveo U280 platform. We implement the quantization and ReSparse algorithms in Python to preprocess the sparse matrix. The conversion into Ultra-CSR and Random-CSR formats is developed in C++ and OpenCL, along with the functionality to transfer the matrices to the HBM memory of the Alveo U280 across its 32 channels. This series of operations—including quantization, ReSparse, format conversion, and data transfer—are collectively referred to as the preprocessing stage. Preprocessing is performed only once on the CPU; once computation begins, all similarity comparisons for query vectors proceed in FPGA without incurring any further preprocessing overhead. Preprocessing the Sparse GloVe dataset takes 203 seconds. To cross-verify the correctness of our onboard implementation’s end-to-end execution, we implement a software version of Top-K SpMV based on Intel *Math Kernel Library* (MKL) [35] and Heap Sort.

As listed in Table I, this study covers experiments with 11 sparse matrices. Due to the absence of publicly available sparse embedding corpora, we employ the sparse embedding techniques proposed by [10] to transform five classic embedding corpora into sparse matrices: Baidu Encyclopedia (Sp. Baidu) [17], Common Crawl for Chinese (Sp. CC_zh) [15], Crawl (Sp. Crawl) [22], GloVe (Sp. GloVe) [26], and Wiki News (Sp. Wiki News) [22]. Additionally, we synthesize six larger sparse matrices with tens of millions of rows to comprehensively validate the performance of the proposed

TABLE I
MATRICES IN THE EVALUATION

Sparse matrices		Rows	Columns	NNZ	Density
Sparsified	Baidu	0.64×10^6	900	8.03×10^7	14.02%
	CC_zh	2×10^6	900	3.82×10^7	2.12%
	Crawl	2×10^6	900	5.56×10^8	30.87%
	GloVe	0.4×10^6	500	2.77×10^7	13.83%
	Wiki News	1×10^6	900	2.74×10^7	3.04%
Uniform		5×10^6	1024	0.99×10^8	1.93%
		10×10^6	768	3.89×10^8	5.07%
		15×10^6	512	5.76×10^8	7.50%
$\Gamma(k=3, \theta=4/3)$		5×10^6	1024	0.96×10^8	1.88%
		10×10^6	768	3.82×10^8	4.98%
		15×10^6	512	5.64×10^8	7.34%

algorithms and hardware design. These matrices have row counts ranging from hundreds of thousands to 15 million, column counts between 500 and 1024, and sparsity levels between 1.88% and 30.87%. Notably, the number of non-zeros per row in synthetic sparse matrices follows a uniform or gamma distribution. To efficiently process these matrices, we encode them using the Ultra-CSR and Random-CSR formats, which reduce memory usage by 82.3% and 56.3% on average, respectively, compared to the COO format.

B. Ablation Study on Approximation Precisions

We conduct a series of ablation experiments to evaluate the impact of 6-bit quantization, multi-core parallelism, and ReSparse approximation methods on the ranking results of Top-K SpMV.

TABLE II
RESPARSE EVALUATION

Sparse matrices		Original NNZ (Density)	ReSparse NNZ (Density)	ReSparse ratio
5M		9.90e7 (1.93%)	7.91e7 (1.55%)	20.1%
10M		3.89e8 (5.07%)	2.93e8 (3.82%)	24.6%
15M		5.76e8 (7.50%)	4.41e8 (5.74%)	23.4%
Sparsified	Baidu	8.03e7 (14.02%)	4.58e7 (8.01%)	42.9%
	CC_zh	3.82e7 (2.12%)	2.97e7 (1.65%)	22.1%
	Crawl	5.56e8 (30.87%)	3.59e8 (19.93%)	35.4%
	GloVe	2.77e7 (13.83%)	1.82e7 (9.08%)	34.4%
	Wiki News	2.74e7 (3.04%)	2.18e7 (2.42%)	20.3%

Based on the ReSparse algorithm, we preprocess all sparse matrices. The results in Table II demonstrate that the ReSparse algorithm effectively reduces the number of non-zeros, with sparsity reductions ranging from 20.1% to 42.9%. On publicly available datasets, it achieves an average compression ratio of 31.0%.

We compare the approximate results with precise CPU-based results and look at values of *K* from 1 to 100. Referring to the first-stage task described in Fig.1, our comparison focuses solely on the proportion of true Top-K row indices falling within the approximate Top-512, denoted as *Precision@K*. We randomly generate 100 vectors for each sparse matrix to mitigate random errors and report the average precision over 100 experiments.

The results of the ablation study on 5 classic embedding corpora are shown in Fig.10, from (a) to (e). We observe

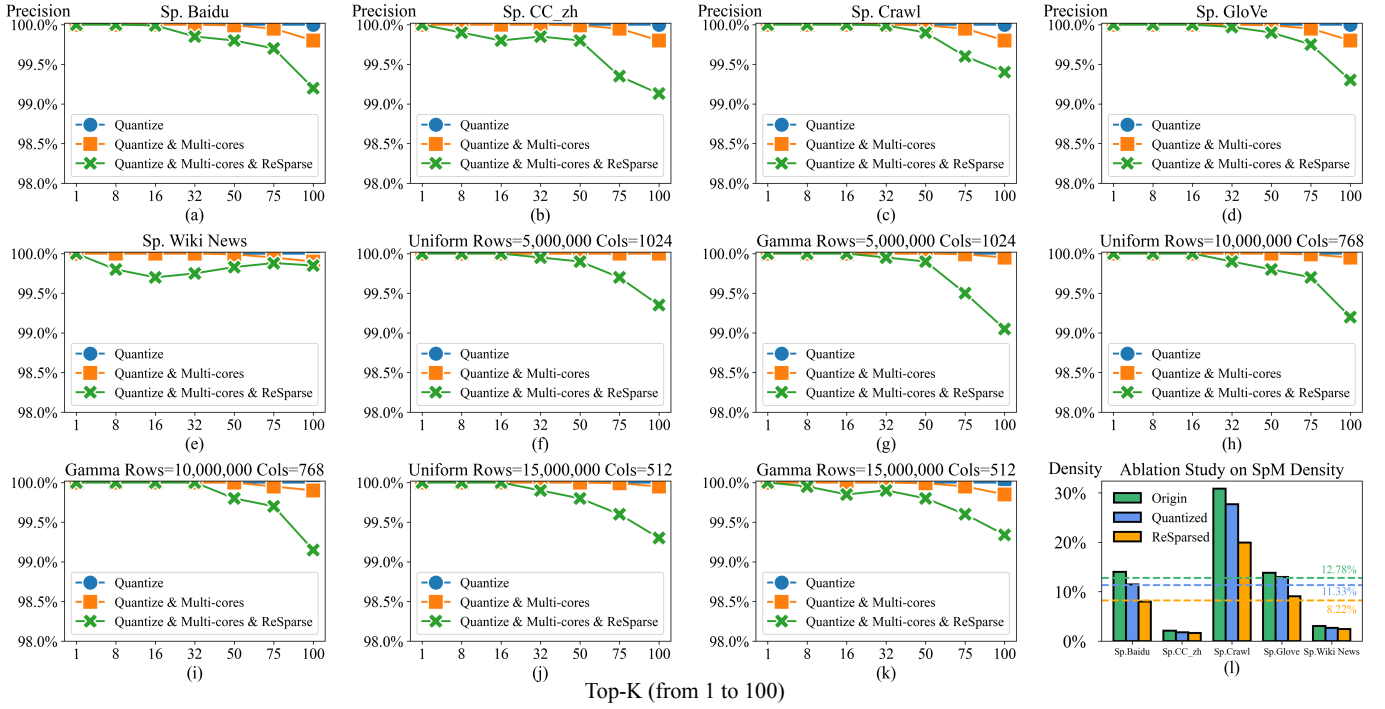


Fig. 10. Ablation study on the impact of Quantization, Multi-cores, and ReSparse on Top-K SpMV *Precision@K* across 11 embedding corpora

the minimal impact of quantization on ranking accuracy. As K increases, the multi-core parallel approximation slightly reduces accuracy, keeping the loss within 0.1%. Incorporating ReSparse technology introduces up to 0.87% loss in *Precision@100*. Fig.10(l) indicates that the quantization process reduces the average density of the sparse matrices from 12.78% to 11.33%. After applying the ReSparse algorithm, the average density further decreases to 8.22%. Conclusively, the average pruning ratio stands at 31.1%.

Fig. 10, spanning from (f) to (k) presents the outcomes of an ablation study conducted on six larger synthetic sparse matrices. The three approximation methods result in a maximum accuracy loss of 0.95%. This indicates that our approximation strategy exhibits robust generalization capabilities for sparse matrices encompassing tens of millions of rows. We find that the multi-core approximation strategy is sensitive to non-zero distributions, offering insights for future improvements.

C. End-to-end Performance

We compare the performance of AcceES based on the Alveo U280 platform with the state-of-the-art sparse embedding similarity accelerator [25]. Additionally, we evaluate the performance of the Top-K SpMV operator on CPU and GPU platforms using *MKL* and *cuSPARSE* for cross-platform comparison. To ensure a fairer comparison, we apply the INT quantization, ReSparse, and multi-cores parallel approximate computing techniques to the CPU and GPU implementations. The configurations of the CPU and GPU are listed in Table III.

Fig. 11 presents the cross-platform computing performance results, revealing that: (1) Both CPU and GPU demonstrate

TABLE III
HARDWARE CONFIGURATION

Platform	CPU Intel Xeon Processor	GPU H100-PCIE-80GB
Compute Units	2.6GHz @ 32 cores	1.75 GHz @ 14592 cores
On-chip Memory	96MB	50MB
Off-chip Memory	238GB/s 256GB DDR4	2TB/s 80GB HBM2e
Library	<i>MKL SpMV, Heap Sort</i>	<i>cuSPARSE INT8, Thrust</i>

sensitivity to the density of sparse matrices. Notably, for matrices with higher density, such as Sparse Crawl, the computational throughput is significantly higher than other datasets. (2) Owing to the parallel granularity of AcceES at the level of non-zeros, the FPGA-based designs effectively mitigate sensitivity to the density of sparse matrices, demonstrating nearly consistent computing performance across matrices of varying sparsity. (3) Our multi-core approximate algorithm, integrated with the ReSparse algorithm, achieves linear speedups on the CPU, delivering an average acceleration of $14.28\times$. The ReSparse algorithm alone contributes an average performance gain of $1.98\times$. On the 5M dataset, ReSparse causes a $3.31\times$ performance drop for the 32-core CPU, as increased sparsity reduces computing efficiency. Ablation experiments on the GPU demonstrate that incorporating INT quantization and the ReSparse software algorithm results in an average performance gain of 24%. Experimental results show that the proposed algorithms are effective on both CPU and GPU architectures, delivering performance gains. (4) In some applications, users only need to know which row vectors will produce the Top-K results, without needing the exact values [3], [6]. In these scenarios, our AcceES-INT6 design fully meets the

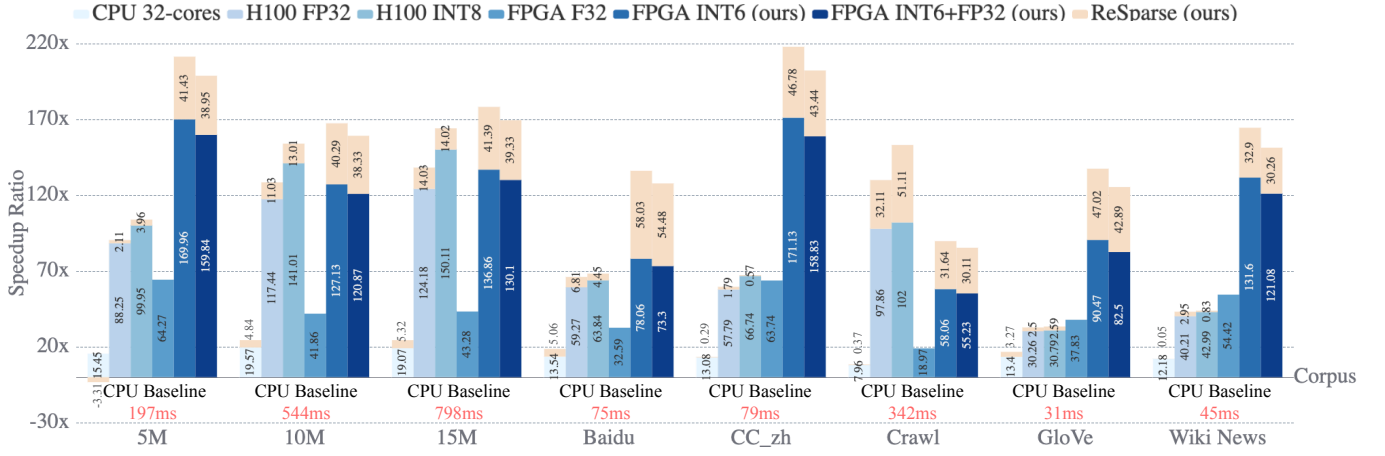


Fig. 11. Cross platform hardware evaluation and ablation study on speedup vs. CPU baseline

requirements and can deliver up to 84.63 GOPS of computational performance. This represents a maximum performance improvement of up to 219.1 $\times$ compared to the CPU. Even compared with the CPU implementation that uses 32-cores and adds the performance gain brought by ReSparse, the average speedup can reach 11.7 $\times$. AccelES-INT6 achieves an average 2.25 $\times$ performance improvement compared to the GPU implementation, which utilizes INT8 quantization and benefits from the additional performance gains provided by ReSparse. Furthermore, the computational throughput of AccelES-INT6 is 2.55 $\times$ of the FPGA-FP32 solution implemented in [25]. Moreover, when integrated with sparsity enhancements from the ReSparse algorithm, AccelES-INT6 attains acceleration ratios of 3.6. 5) In scenarios requiring precise Top-K results, AccelES-FP32 provides a computational throughput of up to 81.16 GOPS, outperforms state-of-the-art CPU-, GPU-, and FPGA-based solutions by 153.3 $\times$, 2.5 $\times$, and 3.4 $\times$ respectively.

D. Scalability and Memory Accesses Reduction

We evaluate the scalability of AccelES-INT6, as depicted in Fig. 12. We implement and evaluate AccelES-INT6 in configurations ranging from 1-core to 32-core, meticulously assessing the computing performance of AccelES-INT6. The findings from our experiments indicate that AccelES-INT6 achieves an almost linear speedup ratio, demonstrating robust scalability.

Fig. 13 illustrates the substantial improvements in memory access performance achieved by AccelES. Compared to the results in [25], AccelES exhibits an impressive average reduction of 73.5% in memory access operations across eight different sparse matrices. This significant decrease in memory access directly correlates with a notable reduction in end-to-end latency, thereby enhancing the overall efficiency and responsiveness of the system.

E. Resource Usage and Power Efficiency

We conduct a comparison with the FPGA-FP32 solution implemented in [25] regarding resources and power consumption, as shown in Table IV.

TABLE IV
RESOURCE USAGE, FREQUENCY, AND POWER EFFICIENCY ANALYSIS

Metrics	FP32 [25]	AccelES-INT6	AccelES-FP32	Total
LUTs	46%	44%	50%	1303680
FFs	39%	38%	43%	2607360
DSPs	18%	13%	14%	9024
BRAM	28%	17%	18%	4032
URAM	46%	63%	65%	960
Freq. (MHz)	203	210	200	-
Power	54W	55W	58W	-
GOPS	33.78	86.43	81.16	-
GOPS/W	0.63	1.57	1.40	-
GOPS/DSPs	0.021	0.071	0.065	-

By enhancing the payload capacity of non-zeros with a 512-bit HBM width, AccelES achieves a corresponding increase in computational parallelism. Accordingly, AccelES utilizes additional on-chip RAM to store multiple instances of x and the results of the SpMV operator computations. Benefitting from our low-bit design, a single DSP can perform three multiplication operations through segmented reuse, resulting in DSP utilization rates of only 13% and 14%. Since a single computing core of AccelES possesses a higher degree of parallelism and requires the configuration of more complex

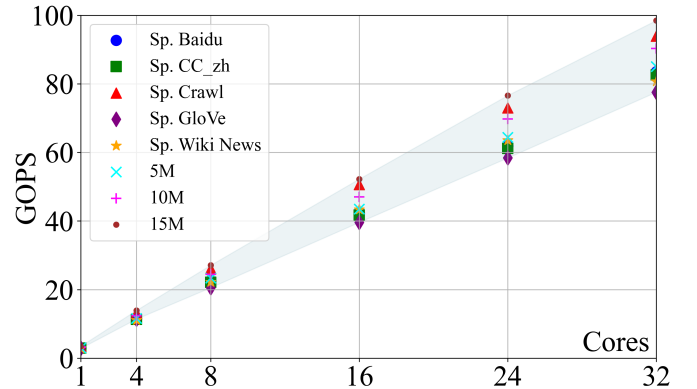


Fig. 12. Scalability of AccelES: computing performance variation with core count

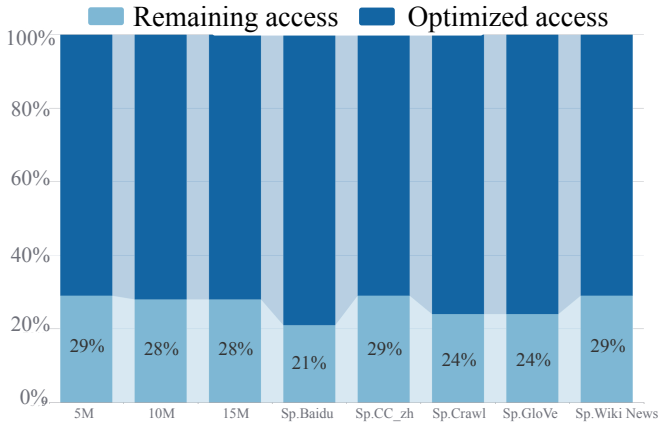


Fig. 13. Performance of the memory access optimization on eight sparse matrices

logic paths and additional resources, the operating frequency of this heterogeneous architecture is set to 200MHz.

TABLE V
CROSS-PLATFORM COMPARISON OF COST EFFICIENCY AND POWER EFFICIENCY

	CPU	GPU	U280
Freq. (MHz)	2600	1755	200
Performance (GOPS)	7.8	60.38	81.16
Power (W)	300	350	58
Cost	\$8,428	\$21,000	\$8,575
Cost Efficiency (MOPS/\$)	0.93	2.88	9.46
Power Efficiency (MOPS/W)	26	172.5	1399.31

We perform cross-platform analyses of cost efficiency and power efficiency, as presented in Table V. Despite the comparable prices of the three platforms, AccelES stands out with the highest cost efficiency. During execution, AccelES consumes only 58W, achieving power efficiency improvements of $53.8\times$ and $8.1\times$ compared to CPU and GPU implementations, respectively.

F. Discussion

1) *Potential on Applications*: Sparse embeddings are widely used in applications like Alibaba’s recommendation system [29], where they process user behavior and product features for real-time recommendations, and in NLP for their interpretability and reduced computational complexity [10]. FastRP [4] demonstrates their effectiveness in graph learning tasks with sparse random projections. AccelES facilitates efficient sparse matrix similarity computations, ensuring accuracy and timeliness of services. Additionally, in *Retrieval-Augmented Generation* (RAG), documents are encoded into fixed-length vectors (typically 768 to 1024 dimensions [8], [27]). AccelES accelerates the computation of similarity scores on these sparse representations, ensuring real-time responsiveness.

2) *Potential on Addressing Top-K SpMSpV*: AccelES can be directly applied to Top-K SpMSpV computations, achieving performance comparable to Top-K SpMV. The key difference lies in vector sparsity. By storing the vector in on-chip URAM

and creating multiple copies, we ensure fast access within a single clock cycle, making this optimization independent of vector sparsity. This allows for efficient memory access in both sparse and dense scenarios, enabling seamless support for Top-K SpMSpV without requiring additional modifications.

3) *Potential to Scale up K*: AccelES provides a high-throughput and high-precision Top-100 SpMV solution, with potential demand for more extensive Top-K results in certain applications. As shown in Fig. 7, AccelES utilizes FPGA computational characteristics to implement a parameter-configurable Top-k Updater. When K exceeds 100 and Top-512 accuracy decreases, parameters can be adjusted to transition from a Top-16 to a Top-32 Updater or higher by increasing pipeline depth and allocating additional on-chip memory resources.

4) *Limitations*: To maximize bandwidth utilization and alleviate on-chip cache pressure, we restrict AccelES to handle sparse matrices with column counts not exceeding 1024. While this limitation may pose challenges in specific applications, it generally meets the requirements in most practical scenarios, effectively enhancing system performance and efficiency.

VIII. CONCLUSION

In high-dimensional data analysis, Top-K SpMV for embedding similarity plays a crucial role. Traditional implementations often struggle to meet stringent real-time requirements, as massive random memory accesses on the matrix elements typically bind them. To address this challenge, we develop AccelES, an approximate Top-K SpMV accelerator that leverages FPGA and HBM with a software-hardware co-design approach.

AccelES utilizes a heterogeneous architecture to streamline the computation process. It begins with low-bit quantization to swiftly identify relevant rows as candidates, then prunes the full-precision dataset based on these candidates. Full-precision calculations are performed on the pruned subset to ensure accuracy. To enhance transmission efficiency, AccelES incorporates new matrix representations, Ultra-CSR and Random-CSR, which improve memory bandwidth utilization by transmitting more valuable data in a single memory access. This design reduces memory access by 73.5%.

The experimental results demonstrate that AccelES delivers a remarkable $11.7\times$ speedup over multi-threaded CPU implementations and exhibits $2.5\times$ higher performance and $8.1\times$ better energy efficiency compared to a GPU with $4.45\times$ higher bandwidth. These achievements underscore the effectiveness of our approach in addressing the stringent demands of real-time applications in high-dimensional data analysis.

ACKNOWLEDGEMENTS

We thank all anonymous reviewers for their insightful comments and valuable feedback. We thank Xuan Peng for the profound discussions on this work. This work is supported in part by the Major Program (JD) of Hubei Province (No.2023BAA024).

REFERENCES

- [1] ing-bank/sparse_dot_topn: Python package to accelerate the sparse matrix multiplication and top-n similarity selection. [Online]. Available: https://github.com/ing-bank/sparse_dot_topn
- [2] Vitis_libraries/sparse/L2/benchmarks at main · xilinx/vitis_libraries. [Online]. Available: https://github.com/Xilinx/Vitis_Libraries/tree/main/sparse/L2/benchmarks
- [3] M. M. Afsar, T. Crump, and B. Far, "Reinforcement learning based recommender systems: A survey," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–38, 2022.
- [4] H. Chen, S. F. Sultan, Y. Tian, M. Chen, and S. Skiena, "Fast and accurate network embeddings via very sparse random projection," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019, pp. 399–408.
- [5] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing spmv on gpu," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2023, pp. 115–128.
- [6] Z. Z. Darban and M. H. Valipour, "Ghrs: Graph-based hybrid recommendation system with application to movie recommendation," *Expert Systems with Applications*, vol. 200, pp. 1–14, 2022.
- [7] J. Dean and S. Ghemawat, "Mapreduce: simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [9] R. Dorrance, F. Ren, and D. Marković, "A scalable sparse matrix-vector multiplication kernel for energy-efficient sparse-blas on fpgas," in *Proceedings of the 2014 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2014, pp. 161–170.
- [10] M. Faruqui, Y. Tsvetkov, D. Yogatama, C. Dyer, and N. A. Smith, "Sparse overcomplete word vector representations," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015, pp. 1491–1500.
- [11] F. Favaro, E. Dufrechou, J. P. Oliver, and P. Ezzatti, "Optimizing the performance of the sparse matrix-vector multiplication kernel in fpga guided by the roofline model," *Micromachines*, vol. 14, no. 11, pp. 1–14, 2023.
- [12] J. Gao, W. Ji, J. Liu, Y. Wang, and F. Shi, "Revisiting thread configuration of spmv kernels on gpu: A machine learning based approach," *Journal of Parallel and Distributed Computing*, vol. 185, pp. 1–13, 2024.
- [13] J. Gao, B. Liu, W. Ji, and H. Huang, "A systematic literature survey of sparse matrix-vector multiplication," *arXiv preprint arXiv:2404.06047*, 2024.
- [14] G. Gerogiannis, S. Aananthakrishnan, J. Torrellas, and I. Hur, "Hottiles: Accelerating spmm with heterogeneous accelerator architectures," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 1012–1028.
- [15] E. Grave, P. Bojanowski, P. Gupta, A. Joulin, and T. Mikolov, "Learning word vectors for 157 languages," in *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, 2018, pp. 1–5.
- [16] F. Incitti, F. Urli, and L. Snidaro, "Beyond word embeddings: A survey," *Information Fusion*, vol. 89, pp. 418–436, 2023.
- [17] S. Li, Z. Zhao, R. Hu, W. Li, T. Liu, and X. Du, "Analogical reasoning on chinese morphological and semantic relations," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 138–143.
- [18] S. Li, D. Liu, and W. L. Liu, "Efficient fpga-based sparse matrix-vector multiplication with data reuse-aware compression," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4606–4617, 2023.
- [19] W. Liu and B. Vinter, "Csr5: An efficient storage format for cross-platform sparse matrix-vector multiplication," in *Proceedings of the 29th ACM on International Conference on Supercomputing (ICS)*, 2015, pp. 339–350.
- [20] D. Loureiro, A. M. Jorge, and J. Camacho-Collados, "Lmms reloaded: Transformer-based sense embeddings for disambiguation and beyond," *Artificial Intelligence*, vol. 305, pp. 1–33, 2022.
- [21] C. Ma, L. Ma, Y. Zhang, H. Wu, X. Liu, and M. Coates, "Knowledge-enhanced top-k recommendation in poincaré ball," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021, pp. 4285–4293.
- [22] T. Mikolov, E. Grave, P. Bojanowski, C. Puhersch, and A. Joulin, "Advances in pre-training distributed word representations," in *Proceedings of the International Conference on Language Resources and Evaluation (LREC)*, 2018, pp. 1–4.
- [23] M. Naumov, L. Chien, P. Vandermersch, and U. Kapasi, "Cuspars library," in *GPU Technology Conference*, 2010.
- [24] J. Ni, J. Li, and J. McAuley, "Justifying recommendations using distantly-labeled reviews and fine-grained aspects," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 188–197.
- [25] A. Parravicini, L. G. Cellamare, M. Siracusa, and M. D. Santambrogio, "Scaling up hbm efficiency of top-k spmv for approximate embedding similarity on fpgas," in *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 799–804.
- [26] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.
- [27] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
- [28] F. Sgherzi, A. Parravicini, and M. D. Santambrogio, "A mixed precision, multi-gpu design for large-scale top-k sparse eigenproblems," in *Proceedings of the 2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 1259–1263.
- [29] Q. Shen, H. Wen, W. Tao, J. Zhang, F. Lv, Z. Chen, and Z. Li, "Deep interest highlight network for click-through rate prediction in trigger-induced recommendation," in *Proceedings of the ACM Web Conference (WWW)*, 2022, pp. 422–430.
- [30] L. Song, Y. Chi, L. Guo, and J. Cong, "Serpens: A high bandwidth memory based accelerator for general-purpose sparse matrix-vector multiplication," in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 211–216.
- [31] L. Song, Y. Chi, A. Sohrabizadeh, Y.-k. Choi, J. Lau, and J. Cong, "Sextans: A streaming accelerator for general-purpose sparse-matrix dense-matrix multiplication," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2022, pp. 65–77.
- [32] W. Stadnik and Z. Nowak, "The impact of web pages' load time on the conversion rate of an e-commerce platform," in *Proceedings of 38th International Conference on Information Systems Architecture and Technology (ISAT)*, 2018, pp. 336–345.
- [33] A. Subramanian, D. Pruthi, H. Jhamtani, T. Berg-Kirkpatrick, and E. Hovy, "Spine: Sparse interpretable neural embeddings," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2018.
- [34] J. D. Trotter, S. Ekmekçi, J. Langguth, T. Torun, E. Düzakın, A. Ilic, and D. Unat, "Bringing order to sparsity: A sparse matrix reordering study on multicore cpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023, pp. 1–13.
- [35] E. Wang, Q. Zhang, B. Shen, G. Zhang, X. Lu, Q. Wu, and Y. Wang, "Intel math kernel library," *High-Performance Computing on the Intel® Xeon Phi™: How to Fully Exploit MIC Architectures*, pp. 167–188, 2014.
- [36] S. Wang, L. Hu, Y. Wang, X. He, Q. Z. Sheng, M. A. Orgun, L. Cao, F. Ricci, and P. S. Yu, "Graph learning based recommender systems: A review," *arXiv preprint arXiv:2105.06339*, 2021.
- [37] Z. Wang, H. Huang, J. Zhang, and G. Alonso, "Shuhai: Benchmarking high bandwidth memory on fpgas," in *Proceedings of the 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2020, pp. 111–119.
- [38] J. Won, C. Mendis, J. S. Emer, and S. Amarasinghe, "Waco: learning workload-aware co-optimization of the format and schedule of a sparse tensor program," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 920–934.
- [39] T. Xia, G. Fu, C. Li, Z. Luo, L. Zhang, R. Chen, W. Zhao, N. Zheng, and P. Ren, "A comprehensive performance model of sparse matrix-vector multiplication to guide kernel optimization," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 2, pp. 519–534, 2022.
- [40] L. Yang, S. Wang, Y. Tao, J. Sun, X. Liu, P. S. Yu, and T. Wang, "Dgrec: Graph neural network for recommendation with diversified embedding generation," in *Proceedings of the 16th ACM International Conference on Web Search and Data Mining (WSDM)*, 2023, pp. 661–669.

- [41] T. Yang, Y. Zhou, Q. Tang, F. Xu, H. Ma, J. Zhao, and L. Jiang, "Spmmplu: A compiler plug-in with sparse ir for efficient sparse matrix multiplication," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [42] M. Yoo, J. Song, J. Lee, N. Kim, Y. Kim, and J. Lee, "Sgcn: Exploiting compressed-sparse features in deep graph convolutional network accelerators," in *Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 1–14.
- [43] Q. Zhang, J. Lu, and Y. Jin, "Artificial intelligence in recommender systems," *Complex & Intelligent Systems*, vol. 7, no. 1, pp. 439–457, 2021.
- [44] X. Zhang, Z. Li, R. Liu, X. Chen, and Y. Han, "Fspa: An fefet-based sparse matrix-dense vector multiplication accelerator," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [45] J. Zheng, J. Jiang, J. Du, D. Huang, and Y. Lu, "Optimizing massively parallel sparse matrix computing on arm many-core processor," *Parallel Computing*, vol. 117, pp. 1–13, 2023.
- [46] L. Zhou, S. Pan, J. Wang, and A. V. Vasilakos, "Machine learning on big data: Opportunities and challenges," *Neurocomputing*, vol. 237, pp. 350–361, 2017.